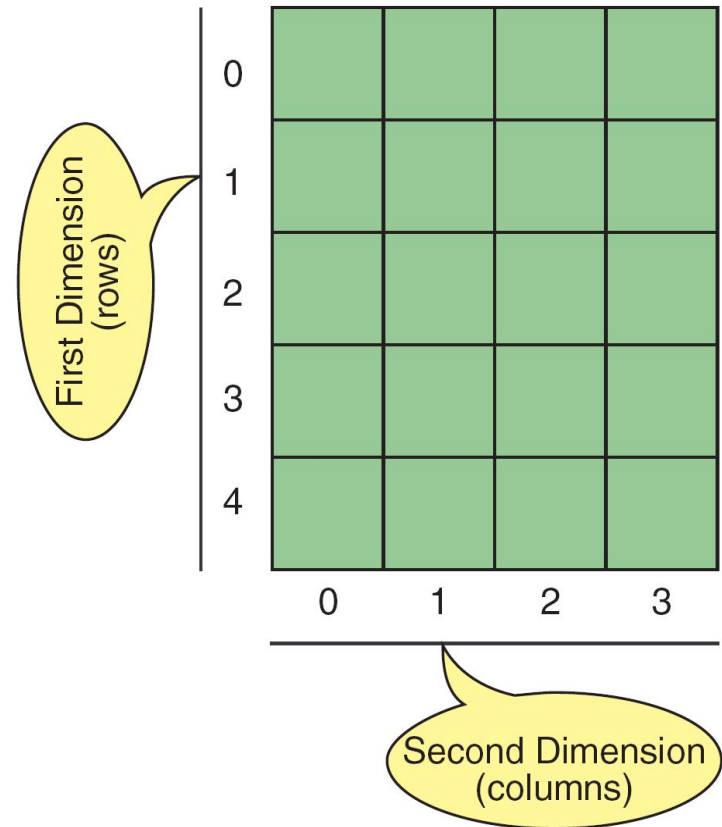


Two-Dimensional Arrays

- Data is often available in tabular form
- Tabular data is often represented in arrays
- Matrix is an example of tabular data and is often represented as a 2-dimensional array



Two-dimensional Array

Two-Dimensional Arrays

- A 2-D array, A with $m \times n$ elements.
- In math application it is called *matrix*.
- In business application – table.
- Example:

Assume 25 students had taken 4 tests.

The marks are stored in 25×4 array locations:

	U0	U1	U2	U3
Stud 0	88	78	66	89
Stud 1	60	70	88	90
Stud 2	62	45	78	88
..	..	..	..	..
..	..	..	..	..
Stud 24	78	88	98	67

n

m

2D Arrays

The elements of a 2-dimensional array **a**
declared as:

int a[3][4];

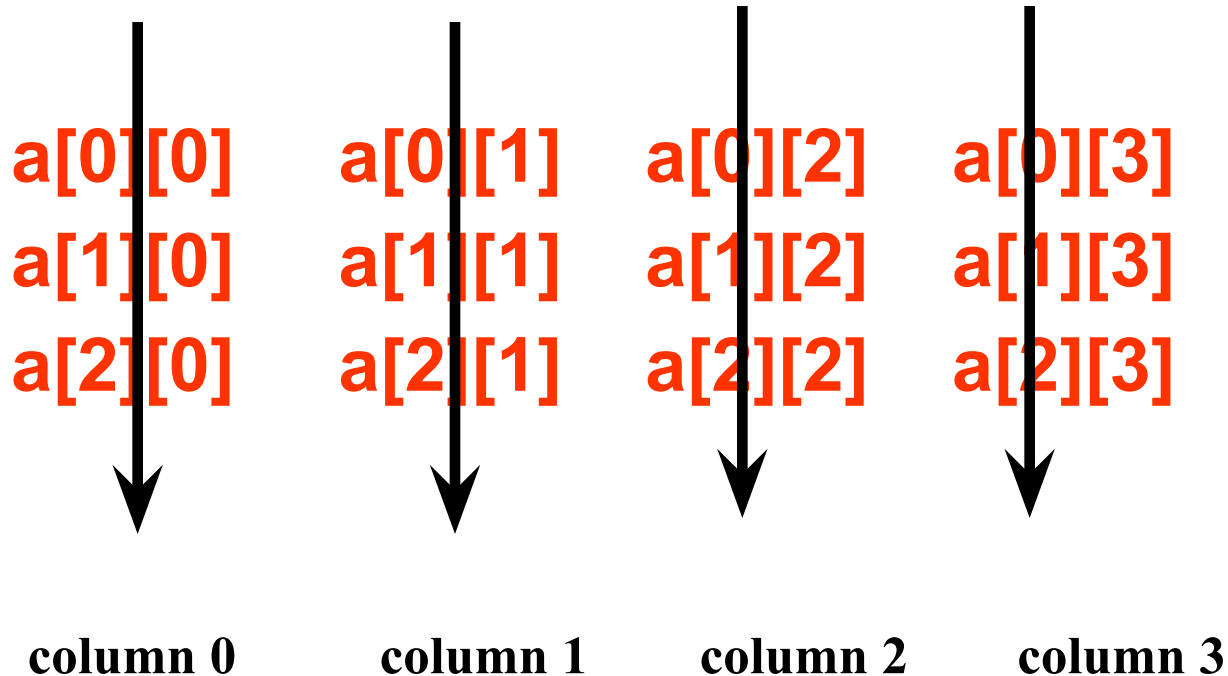
may be shown as a table

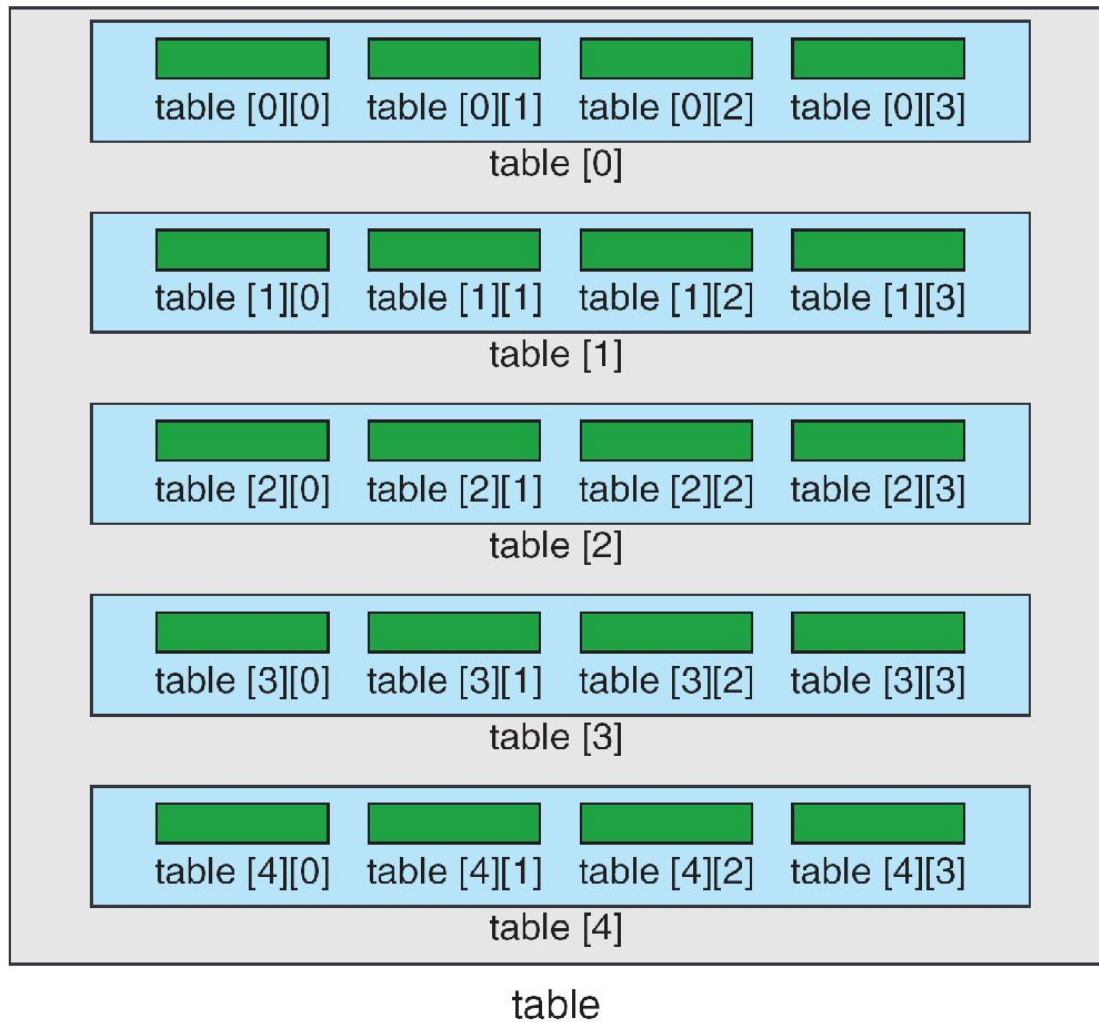
a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

Rows of a 2D Array



Columns of a 2D Array





Array Of Arrays

00	01	02	03	04
10	11	12	13	14

User's View

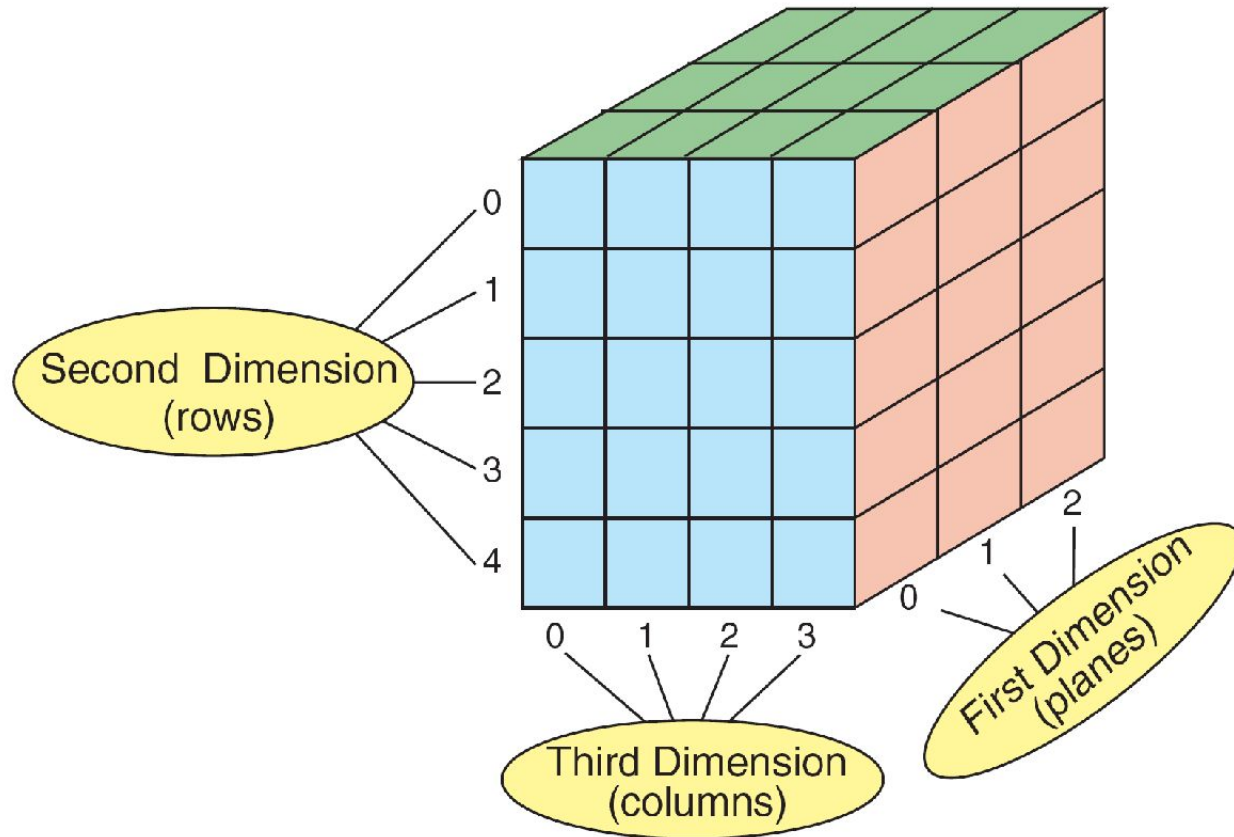
row 0					row 1				
00	01	02	03	04	10	11	12	13	14
[0][0]	[0][1]	[0][2]	[0][3]	[0][4]	[1][0]	[1][1]	[1][2]	[1][3]	[1][4]

Memory View

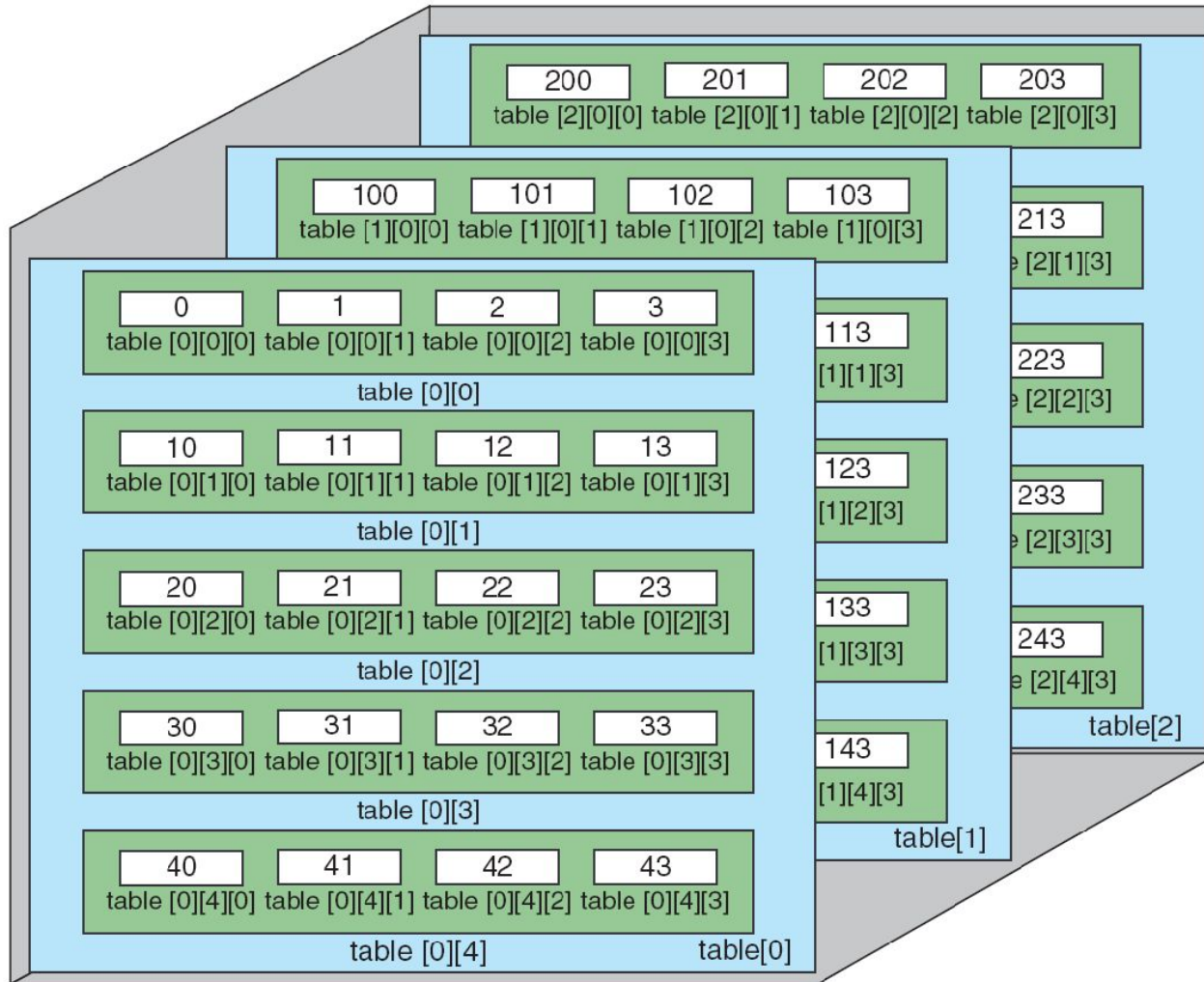
Memory Layout

Multidimensional Arrays

Multidimensional arrays can have three, four, or more dimensions. The first dimension is called a plane, which consists of rows and columns. The C language considers the three-dimensional array to be an array of two-dimensional arrays.



A Three-dimensional Array (3 x 5 x 4)



C View of Three-dimensional Array

Indices in 2D arrays

■ Assume that the two dimensional array called **val** is declared and looks like the following:

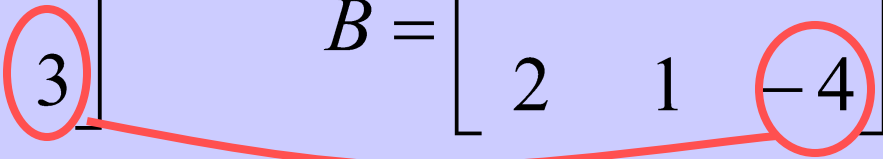
val	Col 0	Col 1	Col 2	Col 3
Row 0	8	16	9	52
Row 1	3	15	27	6
Row 2	14	25	2	10

■ To access the cell containing 6, we reference **val[1][3]**, that is, row 1, column 3.

Addition of Matrices

If A and B are both $m \times n$ matrices then the sum of A and B , denoted $A + B$, is a matrix obtained by adding corresponding elements of A and B .

add these

$$A = \begin{bmatrix} 1 & -2 & 2 \\ 0 & -1 & 3 \end{bmatrix} \quad B = \begin{bmatrix} -3 & 0 & 4 \\ 2 & 1 & -4 \end{bmatrix}$$


$$A + B = \begin{bmatrix} -2 & -2 & 6 \\ 2 & 0 & -1 \end{bmatrix}$$

```
#include<stdio.h>
#include<string.h>
#include<conio.h>

int main() {
    int a[10][10], b[10][10], c[10][10];
    int R, C, i, j;

    printf("Enter Row number: ");
    scanf("%d", &R);

    printf("Enter Column number: ");
    scanf("%d", &C);
```

Matrix Addition (I)

```
printf("Enter First matrix of row=%d, col=%d\n",R,C);  
for(i = 0; i<R; i++){  
    for(j = 0; j<C; j++){  
        scanf("%d",&a[i][j]);  
    }  
}
```

Matrix Addition (II)

```
printf("Enter Second matrix of row=%d, col=%d\n",R,C);  
for(i = 0; i<R; i++) {  
    for(j = 0; j<C; j++) {  
        scanf("%d", &b[i][j]);  
    }  
}
```

Matrix Addition (III)

//Now doing addition

```
for(i = 0; i<R; i++){  
    for(j = 0; j<C; j++){  
        c[i][j] = a[i][j] + b[i][j];  
    }  
}
```

Matrix Addition (IV)

```
//Now printing the Result matrix C
printf("Result is:\n");
for(i = 0; i<R; i++){
    for(j = 0; j<C; j++){
        printf("%d ",c[i][j]);
    }
    printf("\n");
}
return 0;
}
```

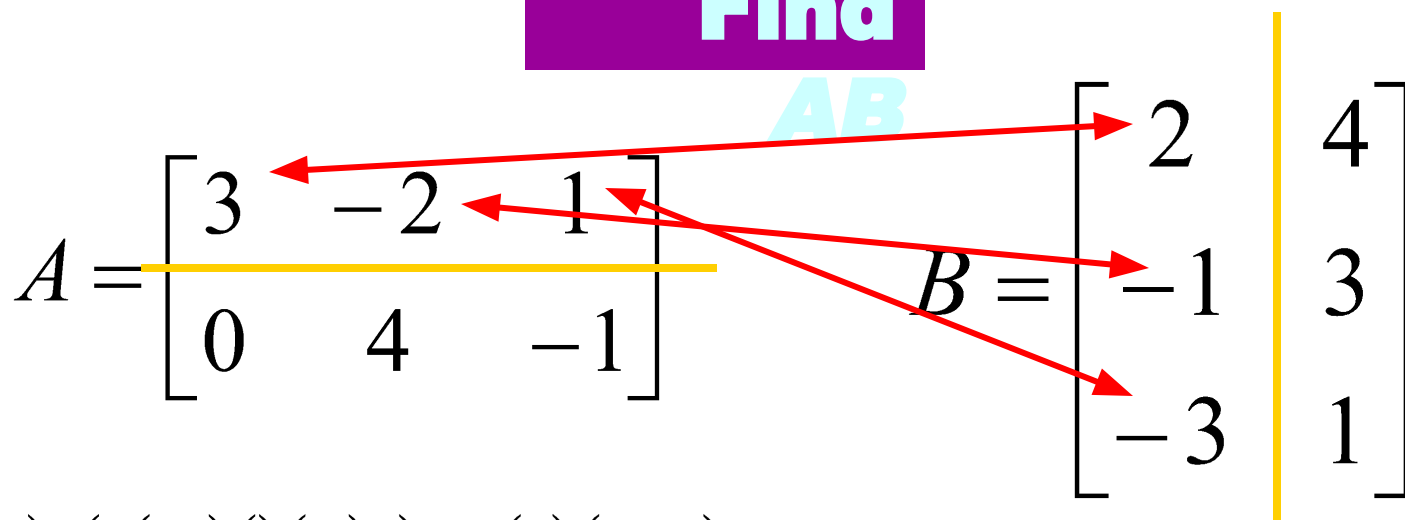
```
Enter Row number: 2
Enter Column number: 3
Enter First matrix of row=2, col=3
1 2 3
3 4 5
Enter Second matrix of row=2, col=3
5 6 7
8 9 10
Result is:
6 8 10
11 13 15
```

Matrix Addition (V)

Multiplication of Matrices

The multiplication of matrices is easier shown than put into words. You multiply the rows of the first matrix with the columns of the second adding products

Find

$$A = \begin{bmatrix} 3 & -2 & 1 \\ 0 & 4 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 4 \\ -1 & 3 \\ -3 & 1 \end{bmatrix}$$


$$(3)(2) + (-2)(-1) + (1)(-3) = 5$$

First we multiply across the first row and down the first column adding products. We put the answer in the first row, first column of the answer.

Find

AB

$$A = \begin{bmatrix} 3 & -2 & 1 \\ 0 & 4 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 4 \\ -1 & 3 \\ -3 & 1 \end{bmatrix}$$

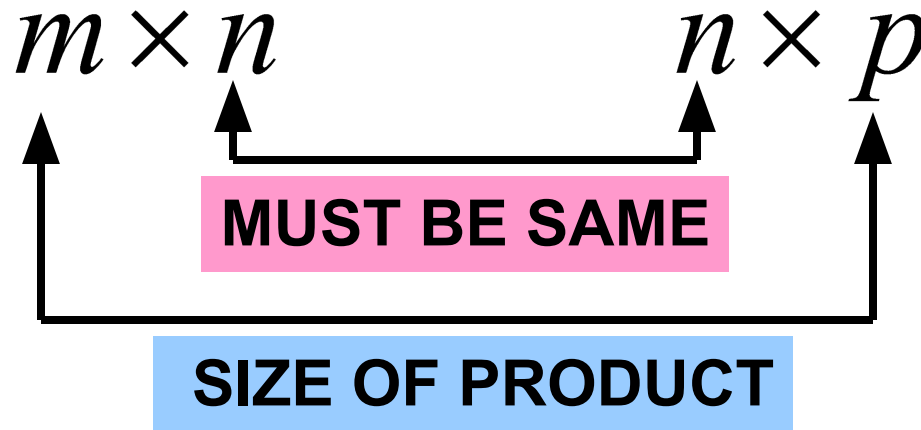
$$AB = \begin{bmatrix} 5 & 7 \\ -1 & 1 \end{bmatrix}$$

~~$(0)(2) + (4)(-1) + (-1)(-3) = 0 - 4 + 3 = -1$~~

Notice the sizes of A and B and the size of the product AB .

Now we multiply across the **second** row and down the **second** column and we'll put the answer in the second row, second column.

To multiply matrices A and B look at their dimensions



If the number of columns of A does not equal the number of rows of B then the product AB is undefined.

```
#include<stdio.h>
int main() {
    int a[5][5],b[5][5],c[5][5],i,j,k,sum=0,m,n,o,p;
    printf("\nEnter the row and column of first matrix");
    scanf("%d %d",&m,&n);
    printf("\nEnter the row and column of second matrix");
    scanf("%d %d",&o,&p);
    if(n!=o) {
        printf("Matrix mutiplication is not possible");

    }
```

Matrix Multiplication (I)

```
else{
    printf("\nEnter the First matrix->");
    for(i=0;i<m;i++)
    for(j=0;j<n;j++)
        scanf("%d",&a[i][j]);
    printf("\nEnter the Second matrix->");
    for(i=0;i<o;i++)
    for(j=0;j<p;j++)
        scanf("%d",&b[i][j]);
```

Matrix Multiplication (II)

```
for (i=0; i<m; i++)
    for (j=0; j<p; j++)
        c[i][j]=0;
for (i=0; i<m; i++) { //row of first matrix
    for (j=0; j<p; j++) { //column of second matrix
        sum=0;
        for (k=0; k<n; k++)
            sum=sum+a[i][k]*b[k][j];
        c[i][j]=sum;
    }
}
```

Matrix Multiplication (III)

```
printf("\nThe multiplication of two matrix
is\n");
    for (i=0;i<m;i++) {
        printf("\n");
        for (j=0;j<p;j++) {
            printf("%d\t",c[i][j]);
        }
    }
    return 0;
}
```

Matrix Multiplication (IV)

Row-wise and Column-wise sum

```
#include<stdio.h>
#include<string.h>
#include<conio.h>
```

```
void main ()
{
    int array[10][10];
    int i, j, m, n, sum = 0;

    printf("Enter the order of the matrix\n");
    scanf("%d %d", &m, &n);
```

Taking input (I)

```
for (i = 0; i < m; ++i)
{
    for (j = 0; j < n; ++j)
    {
        scanf("%d", &array[i][j]);
    }
}
```

Taking input (II)

```
for (i = 0; i < m; ++i)
{
    for (j = 0; j < n; ++j)
    {
        sum = sum + array[i][j] ;
    }
    printf("Sum of the %d row is = %d\n", i, sum);
    sum = 0;
}
```

Row-wise sum(III)

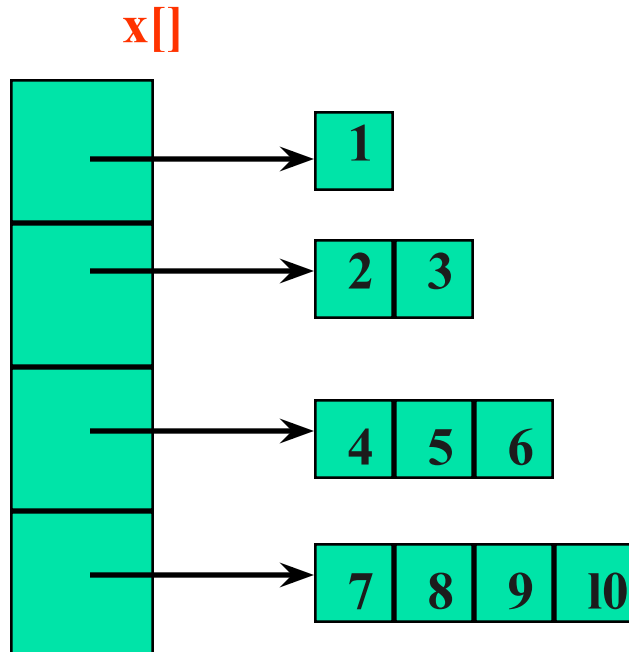
```
sum = 0;
for (j = 0; j < n; ++j)
{
    for (i = 0; i < m; ++i)
    {
        sum = sum + array[i][j];
    }
    printf("Sum of the %d column is = %d\n", j, sum);
    sum = 0;
}
}
```

Column-wise sum(IV)

-
- Matrix Multiplication

Assignment on Matrices

Irregular 2D Arrays



Irregular 2-D array: the length of rows is not required to be the same.